

Cross-Site Scripting (XSS): Impacto Real e Mitigação Eficaz

Miguel Pombeiro e Miguel Rocha

Universidade de Évora, Évora, Portugal
{157829,158501}@alunos.uevora.pt

Resumo Desde a sua descoberta no final da década de 1990, o *Cross-Site Scripting* (XSS) tem-se revelado uma grande ameaça às aplicações *web*, comprometendo a privacidade, segurança e integridade dos dados dos utilizadores. O XSS é uma vulnerabilidade de aplicações *web*, em que *scripts* maliciosos são injetados nos *websites* confiáveis, o que permite aos atacantes executar código nos *browsers* das vítimas.

Para compreender a complexidade desta ameaça, este artigo apresenta a definição do ataque e do seu ciclo de vida, desde a identificação de vulnerabilidades até à sua exploração. São ainda abordados os três principais tipos de ataques, nomeadamente o *Stored XSS*, *Reflected XSS* e *DOM-based XSS*, sendo explorados tanto os seus vetores de ataque como os seus ciclos de vida. Posteriormente, e de modo a analisar as consequências reais do XSS, examina-se o seu impacto quer para os utilizadores quer para os sistemas e organizações, sendo descritos alguns incidentes reais de cibersegurança. Por fim, são discutidas algumas das estratégias de defesa e mitigação propostas pela comunidade científica. Esta avaliação engloba não só as boas práticas preventivas de desenvolvimento, mas também os mecanismos ativos de bloqueio em *runtime*, evidenciando a necessidade de uma defesa em múltiplas camadas.

Palavras-chave: Cross-Site Scripting · XSS · Cibersegurança · Defesa · Aplicações Web · Impacto · Mitigação

1 Introdução

O rápido crescimento dos serviços *online* – desde *e-commerce*, bancos até às redes sociais – tem transformado as aplicações *web* em ferramentas essenciais para a sociedade, mas esta evolução tem, ao mesmo tempo, introduzido muitos desafios ao nível da cibersegurança [3] [8]. Dentre estas ameaças, uma das mais críticas e prevalentes é o **Cross-Site Scripting (XSS)**, um ataque de injeção de código do lado do cliente, onde os adversários injetam *scripts* maliciosos, tipicamente escritos em JavaScript. Esta injeção ocorre em páginas *web* confiáveis visualizadas por utilizadores desatentos [1] [3].

As vulnerabilidades dos ataques XSS provêm, fundamentalmente, da insuficiente validação de *inputs* e de uma fraca higienização/sanitização dos dados [3]. Quando uma aplicação *web* aceita *input* de utilizadores sem verificar o seu formato ou realizar qualquer tipo de neutralização, pode estar, inadvertidamente,

a deixar passar código malicioso [7]. O *browser* da vítima, que não consegue distinguir o *script* injetado do código legítimo da aplicação, acaba por executar o *payload* [3]. Isto permite aos atacantes iludirem e contornarem a ***Same Origin Policy (SOP)***, que é um mecanismo de segurança fundamental dos *browsers*, usado para prevenir que documentos ou *scripts* de uma origem interajam com recursos de uma origem diferente [8].

Apesar deste tipo de ataques ter sido descoberto entre o final da década de 1990 e o início dos anos 2000, o XSS continua, atualmente, a ser uma praga altamente relevante e difundida na internet [5].

Nas últimas duas décadas, o *Open Web Application Security Project (OWASP)* tem, consistentemente, classificado o XSS no top dez dos riscos de segurança mais críticos das aplicações *web* [3] [8]. O XSS continua tão relevante que ainda representa cerca de 37% a 40% de todas as tentativas de ataque a aplicações *web* [8]. Além disso, estudos realizados em 2023 e 2024 estimavam que cerca de 60% a 65% dos *websites* a nível global continham vulnerabilidades ao XSS [1] [8].

As consequências de um ataque XSS bem-sucedido podem ser bastante severas. Um atacante consegue roubar contas e *cookies* de sessão, alterar conteúdos dos *websites*, distribuir *malware* ou mesmo aceder a informação pessoal e financeira [1] [3]. Estas consequências vão, certamente, ter impacto tanto financeiro como na reputação das organizações. Desta forma, a defesa e mitigação de ataques XSS representa um desafio crescente, pois novos vetores de injeção surgem constantemente. Tal deve-se à adoção, por parte das aplicações, de tecnologias cada vez mais complexas, como HTML5, AJAX e APIs externas [5] [8].

2 Caracterização dos Ataques XSS

O Cross-Site Scripting (XSS), como já foi referido, é um ataque de injeção de código do lado do cliente que atinge a camada de aplicação de um sistema *web* [1] [3]. A tecnologia por trás deste ataque tira partido de falhas críticas não só na validação, bem como na higienização/sanitização dos dados inseridos pelos utilizadores [3] [7]. Na prática, quando uma aplicação *web* aceita e exibe dados não confiáveis sem verificar o seu tipo, formato ou conteúdo, abre a porta para que vetores maliciosos (*payloads*) sejam injetados em páginas *web* legítimas [3]. Esta ameaça sucede porque o *browser* da vítima não consegue distinguir o conteúdo malicioso injetado pelo atacante do código legítimo, acabando por processar e executar a totalidade do código de forma despercebida e dentro do domínio de confiança da aplicação [3] [4].

A superfície de ataque do XSS aumentou drasticamente com o tempo, devido à adoção generalizada de *Single Page Applications* (SPAs) e à forte dependência de APIs que trocam dados complexos de forma assíncrona e que, muitas vezes, contribuem para manipulações dinâmicas e contínuas do *Document Object Model* (DOM) [3] [5]. Se, por um lado, estas tecnologias melhoram a fluidez e a experiência do utilizador, por outro, multiplicam os pontos onde a injeção de *scripts* pode ocorrer no lado do cliente.

Para compreender a verdadeira dimensão e eficácia destas injeções na arquitetura *web* atual, é necessário entender como esta ameaça acontece e se propaga. A exploração de uma vulnerabilidade XSS não ocorre num único instante, mas sim, através de um processo metódico e sequencial [4]. É precisamente a análise técnica e sequencial destes passos operacionais que define o ciclo de vida de um ataque XSS [4].

2.1 O Ciclo de Vida de um ataque XSS

O principal processo deste ataque consiste em explorar uma falha da aplicação *web* na validação, filtração ou sanitização dos dados vindos do utilizador antes de os renderizar no *browser* [3] [8]. Enquanto os seus mecanismos exatos podem variar consoante o tipo a que pertencem – tipos estes explorados na Subsecção 2.2 –, o seu ciclo de vida mantém-se consistente entre eles. O ciclo de vida de um ataque XSS pode ser dividido em quatro fases principais.

Fase 1: Identificação de vulnerabilidades e injeção do *payload*

Este ciclo começa com a identificação de uma aplicação *web* vulnerável que aceita dados vindos do utilizador sem os verificar [8]. Ao descobrir esta falha, o atacante cria um *payload* malicioso, tipicamente escrito em JavaScript (mas também pode utilizar HTML, VBScript, entre outros) [8]. Por fim, o atacante injeta o *script* na aplicação vulnerável, quer seja ao guardar na base de dados do servidor (e.g. num campo de comentários ou numa publicação de fórum) ou incorporar num URL feito à medida [2] [8].

Fase 2: Entrega à vítima

Nesta fase, um utilizador ingénuo acaba por interagir com a aplicação comprometida, quer por navegar na *internet*, quer por ser vítima de um ataque de engenharia social. Neste último, a vítima é convencida a clicar em *links* maliciosos, criados pelo atacante [8] [12].

Fase 3: Execução do *payload*

O servidor *web* vulnerável, processa o pedido e devolve a resposta HTTP, que contém o *script* malicioso injetado [4] [12]. Como o *browser* da vítima não consegue distinguir este *script* do código legítimo da aplicação, acaba por confiar no *payload* e executa-o dentro do contexto de segurança do *site* vulnerável [8].

Fase 4: Exploração e roubo de dados

Por fim, o *script* malicioso executa ações não autorizadas em segundo plano, sem o utilizador saber. Normalmente, o *script* acede a dados sensíveis, como *cookies*, *session tokens* ou *passwords*, que são depois transmitidos para o servidor do atacante [4] [12]. Com estes dados, o atacante consegue sequestrar a sessão para se camuflar como o utilizador legítimo, roubar dados financeiros e pessoais, ver as teclas pressionadas pela vítima, ou até mesmo redirecionar o *browser* para *sites* maliciosos [4] [13].

2.2 Tipos de Ataques

Tradicionalmente, as vulnerabilidades XSS podem ser classificadas em três tipos principais, de acordo com a forma como o *payload* é processado e entregue: *Stored XSS*, *Reflected XSS* e *DOM-based XSS* [3] [7]. No entanto, à medida que as tecnologias *web* evoluíram, outras variantes mais complexas e avançadas surgiram (e.g. Mutation XSS), que deram origem a uma classificação mais abrangente, proposta por Hannousse *et al.* [5], mas que não iremos explorar neste artigo.

Stored XSS

O *Stored XSS* ou XSS persistente (Figura 1) foi considerado uma das formas mais perigosas de XSS [9] [10]. Neste tipo de ataques, o atacante consegue injetar um *script* malicioso, que é guardado de forma permanente na base de dados, no sistema de ficheiros ou no repositório da aplicação [3] [5]. Os alvos normais deste tipo de ataque são plataformas interativas como as redes sociais, campos de comentários, fóruns, entre outros [3]. Ao contrário de outros tipos de XSS, *Stored XSS* não precisa de engenharia social para chegar às suas vítimas, em vez disso, o *payload* fica integrado na aplicação *web* legítima [3]. Quando um utilizador navega para uma página infetada, o servidor *web* envia para o *browser* tanto informação legítima como os *scripts* maliciosos, que acabam por ser executados automaticamente. Como o *payload* está armazenado no servidor, uma única vulnerabilidade XSS explorada pode acabar por infetar um grande número de utilizadores [3] [9] [10].

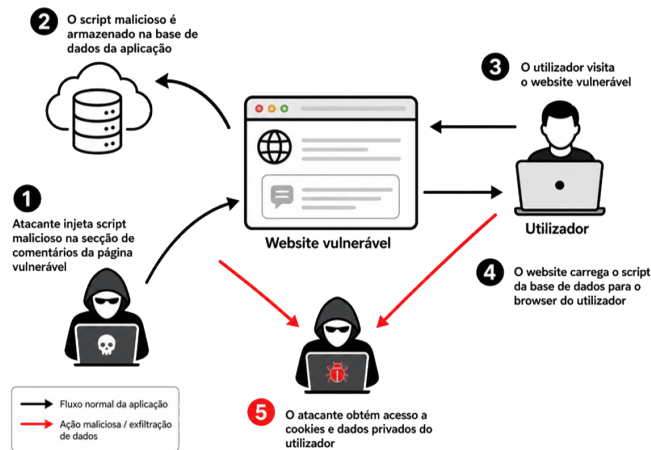


Figura 1. Ciclo de vida do *Stored XSS*.

Reflected XSS

Reflected XSS ou XSS não persistente (Figura 2) é uma das formas de XSS mais comuns [2]. Este tipo de ataque é mais direcionado e depende fortemente

de técnicas de engenharia social [9]. Ao contrário do *Stored XSS*, em ataques de *Reflected XSS* os *scripts* maliciosos não são armazenados no servidor, estão integrados diretamente num URL ou num componente de *input* [10]. Quando a vítima clica no *link*, o pedido é enviado para o servidor vulnerável, que depois, reflete o *script* malicioso para o *browser* do utilizador, dentro da resposta HTTP (e.g., dentro de uma mensagem de erro dinâmica) [7] [12]. Desta forma, o *browser* acaba por executar o *script* pensando que faz parte do código legítimo da página [10]. Este ataque é temporário e, normalmente, apenas afeta o utilizador que clicou no *link* [3].

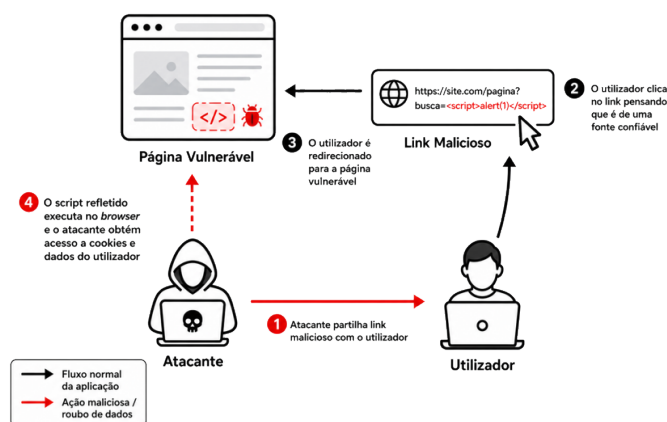


Figura 2. Ciclo de vida do *Reflected XSS*.

DOM-Based XSS

Ao contrário do *Reflected* e *Stored XSS*, o *Document Object Model* (DOM)-based XSS (Figura 3) é uma vulnerabilidade avançada que ocorre exclusivamente no lado do cliente [3] [10] [13]. Neste ataque, o *script* malicioso não necessita de ser processado nem refletido pelo servidor. Em vez disso, este ataque explora *scripts* do lado do cliente (normalmente escritos em JavaScript, AJAX ou jQuery) com a capacidade de alterar a estrutura do DOM de forma dinâmica [8]. Um exemplo desta exploração ocorre através da manipulação indevida de propriedades e objetos de navegação, como o `window.location` ou `document.location`, onde a aplicação extrai dados manipulados diretamente do URL e os processa de forma insegura [4] [12]. Como o ataque ocorre inteiramente no cliente e os dados maliciosos podem nem chegar ao servidor (estando frequentemente ocultos após o identificador # do URL), o ataque consegue contornar muito facilmente as tradicionais ferramentas de segurança [3] [10] [12].

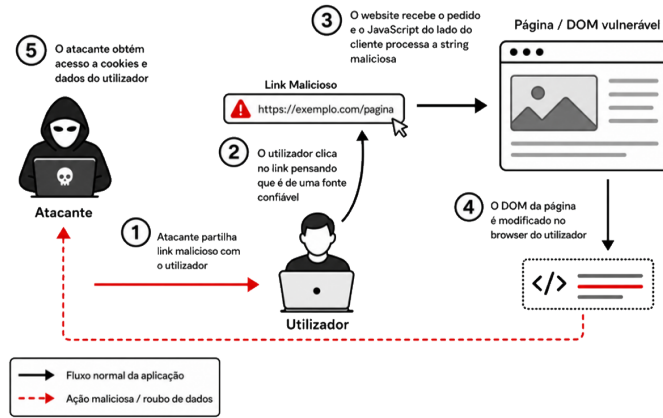


Figura 3. Ciclo de vida do *DOM-based XSS*.

3 Impacto no Mundo Real

Tendo estabelecido a base teórica do XSS, incluindo o seu ciclo de vida e os seus diferentes tipos, é crucial examinar as implicações práticas desta vulnerabilidade. Com a dependência atual da sociedade em relação às aplicações *web* – ferramentas que se tornaram essenciais para áreas críticas como as finanças ou o *e-commerce* –, as consequências de uma injeção de código bem-sucedida estendem-se além do ecossistema do *browser* [3] [8].

Apesar de ser uma ameaça documentada há mais de duas décadas, um ataque bem-sucedido compromete a confidencialidade de informações sensíveis, permitindo o roubo de credenciais de acesso, de dados pessoais, entre outros, o que resulta muitas vezes em repercussões não só para indivíduos como para organizações [5] [13].

3.1 Impacto nos Utilizadores

A execução bem-sucedida de um ataque XSS afeta principalmente a privacidade e a segurança do utilizador final, já que este opera frequentemente de forma silenciosa no seu ambiente local. O principal objetivo destes ataques é a extração de dados confidenciais, como palavras-passe, números de cartões de crédito, outros detalhes bancários e *cookies* de sessão [3] [4] [8].

Com o roubo destes dados pessoais, os atacantes conseguem roubar a identidade do utilizador, bem como realizar o sequestro de sessão (*session hijacking*), assumindo a identidade da vítima para executar ações não autorizadas em seu nome [3] [13]. Para além disto, os atacantes também podem recorrer a técnicas de *keylogging*, que registam todas as teclas pressionadas pelo utilizador durante a sua navegação, capturando novas credenciais [4].

Em casos mais severos, o XSS serve como vetor inicial para forçar o *browser* a transferir *Trojans* e a instalar *malware* na máquina da vítima [2]. No limite, esta infecção pode transformar o computador do utilizador num nó de uma *botnet* utilizada para executar ataques de *Distributed Denial of Service* (DDoS) contra outros alvos na Internet [4].

3.2 Impacto nos Sistemas e Organizações

Por outro lado, as consequências para os sistemas e organizações são igualmente catastróficas. Apesar de a injeção e a execução destes tipos de ataques ocorrerem sempre do lado do cliente, a verdadeira responsabilidade e o dano nos sistemas recaem sobre as entidades que não protegeram devidamente as suas aplicações *web*.

O XSS permite aos atacantes alterarem dinamicamente o conteúdo e a aparência das páginas que são apresentadas – *virtual defacement* –, o que resulta na inutilização dos serviços, na exibição de propaganda maliciosa, entre outros [12] [13]. Para além disso, o impacto para estas organizações engloba perdas financeiras e danos reputacionais. Estas perdas financeiras surgem não só da necessidade de reparação técnica de *software*, mas também dos custos associados à resposta a estes incidentes [13].

Complementarmente, quando os utilizadores finais perdem as suas poupanças ou veem os seus dados sensíveis expostos devido a vulnerabilidades numa aplicação que consideravam fiável, as marcas enfrentam uma forte degradação da sua reputação no mercado, o que se pode traduzir na perda de clientes e em graves prejuízos [13].

3.3 Casos Reais Conhecidos

Ao analisar casos reais conhecidos, é possível observar a verdadeira escala e gravidade destas vulnerabilidades. Pelos vários acontecimentos descritos na literatura, pode-se dizer que todas as organizações – independentemente do seu tamanho ou dos recursos gastos em cibersegurança –, são suscetíveis de sofrer ataques de XSS. Os casos descritos nesta secção exemplificam como o XSS foi utilizado para atacar plataformas de grande dimensão ao longo dos anos.

***Samy Worm* no MySpace (2005)**

Um dos ataques de XSS mais significativos foi o "*Samy*" *Worm*, que teve como alvo a rede social MySpace em 2005 [4] [12]. Samy Kamkar conseguiu arranjar forma de ultrapassar os filtros anti-XSS da plataforma e injetar no seu próprio perfil um *payload* de JavaScript [12]. Quando uma pessoa visitava o perfil comprometido, o *script* obrigava automaticamente o *browser* da vítima a adicionar o Samy como amigo e copiava o *payload* para o perfil da vítima [12]. Este *worm* de *Stored XSS*, foi bastante agressivo e conseguiu replicar-se de forma exponencial, infetando aproximadamente um milhão de utilizadores do MySpace em menos de 24 horas [4] [12]. Para resolver a situação, a empresa teve de desativar temporariamente a aplicação para erradicar o código malicioso [4] [12].

Seller Account Hijacking no eBay (2015-2016)

Entre o final de 2015 e o começo de 2016, o eBay encontrou uma vulnerabilidade crítica de XSS relacionada a um parâmetro de URL mal validado [13]. Os atacantes conseguiram injetar código malicioso nas páginas do eBay, garantindo o acesso completo a contas de vendedores [13]. Com este acesso, conseguiram roubar detalhes de pagamento, bem como realizar vendas fraudulentas não autorizadas a preços com desconto [13]. Embora o eBay tenha eventualmente corrigido a vulnerabilidade, os ataques continuaram até 2017, tendo impacto mesmo após a sua correção [13].

Magecart Attack na British Airways (2018)

Um ótimo exemplo de como o XSS pode ser utilizado para cometer grandes roubos financeiros foi o ataque sofrido pela British Airways em 2018 às mãos do grupo de *hackers* conhecido como Magecart [13]. Os atacantes exploraram uma vulnerabilidade de XSS na biblioteca Feedify de JavaScript utilizada no *website* da empresa [13]. Através da alteração deste *script*, os atacantes conseguiram transmitir os dados dos clientes, de forma silenciosa, para um servidor [13]. Apesar deste servidor falso ter um certificado SSL e apenas simular o *site* da British Airways, o ataque só foi detetado depois de terem sido roubadas informações de mais de 380 000 transações [13].

4 Defesa e Mitigação de Ataques XSS

Face à gravidade e ao impacto sistémico dos incidentes de XSS ao longo de mais de duas décadas, abordados na Secção 3, é possível comprovar que não existe uma solução única e infalível para eliminar totalmente esta vulnerabilidade [7]. Como a esmagadora maioria destas falhas tem origem em erros humanos, na ausência de familiaridade com segurança por parte dos programadores e na adoção de práticas de desenvolvimento de *software* inseguras, a mitigação eficaz do XSS exige uma atenção rigorosa na forma como as aplicações *web* interagem e processam os dados fornecidos pelos utilizadores [5] [8].

Atualmente, a literatura científica já reconhece que os mecanismos tradicionais, baseados unicamente em simples filtros de *blacklists* ou na revisão manual de código, já não são suficientes para acompanhar a complexidade crescente das arquiteturas *web* modernas [3] [5]. Assim, para proteger os sistemas de forma estruturada, o combate ao XSS deve ser distribuído por várias camadas de segurança que se devem complementar [7] [12].

4.1 Técnicas de Prevenção e Mitigação

Numa primeira fase, durante o desenvolvimento do *software*, é importante atuar na raiz do problema, ou seja, implementar medidas de prevenção como a validação de *inputs* e a codificação ou higienização adequada dos *outputs* (incluindo a sanitização do DOM no lado do cliente) antes de exibir o conteúdo no *browser* [4] [5] [6] [11]. Em seguida serão discutidas algumas das técnicas mais utilizadas para prevenir e mitigar o XSS.

Validação de *Input* e Codificação de *Output*

Um dos princípios mais importantes na prevenção de ataques consiste em não confiar nos dados de *input* fornecidos pelo utilizador [3]. Assim, a literatura recomenda a sua validação com base em *whitelists* (em detrimento de *blacklists*), que permitem apenas formatos e caracteres explicitamente autorizados [5] [12]. Complementarmente, também deve ser feita a codificação de *output*, para converter caracteres com significado sintático em entidades HTML seguras [12]. Desta forma, esta dupla camada de proteção assegura que, mesmo que dados manipulados entrem no sistema, o *browser* irá tratá-los estritamente como conteúdo de texto inofensivo e não como código executável.

Uso de *Templates* Seguros

Os *templates web* seguros estão, normalmente, integrados em *frameworks* modernos [5]. Estes *templates* têm mecanismos de auto-sanitização já implementados e, assim, abstraem a complexidade do tratamento direto dos dados [5]. Como a sanitização é feita automaticamente, grande parte das vulnerabilidades, que têm origem em esquecimentos ou erros na higienização manual do código, fica resolvida.

APIs Seguras

Para mitigar ataques complexos como o *DOM-based XSS*, é importante desenvolver APIs seguras que restrinjam chamadas diretas a funções vulneráveis do *browser* (e.g. `eval()`) [5].

Sanitização do Lado do Cliente

A utilização de bibliotecas de sanitização do lado do cliente, como o DOMPurify, é importante [6]. Estas ferramentas filtram strings HTML e objetos diretamente no DOM, removendo atributos e *tags* perigosos de forma fiável antes de as apresentar no ecrã [6].

4.2 Técnicas de Defesa em *Runtime*

Devido à enorme complexidade das aplicações *web* modernas, a prevenção no código nunca é totalmente infalível, logo as medidas de mitigação devem ser obrigatoriamente reforçadas por mecanismos de defesa ativos em *runtime* [5] [8]. A utilização de políticas de segurança rigorosas enviadas pelo servidor (como a *Content Security Policy*) e a filtragem e inspeção de tráfego são mecanismos vitais, já que bloqueiam a execução de *scripts* não autorizados em tempo real, mesmo que uma falha no código tivesse permitido a injeção do *payload* [5]. Estas e outras técnicas de defesa serão discutidas em seguida.

Content Security Policy (CSP)

A *Content Security Policy* é uma norma de segurança transmitida pelo servidor através de cabeçalhos HTTP, que restringe as origens a partir das quais o navegador pode carregar e executar recursos [5]. Aqui são definidas *whitelists* rigorosas para *scripts* e domínios confiáveis [5] [10]. Desta forma, o *browser* bloqueia ativamente a execução de código não autorizado e apenas permite o carregamento de

recursos a partir dos domínios especificados [10]. Assim, mesmo que ocorra uma injeção de código, o *payload* malicioso não é executado e não consegue enviar dados para servidores de terceiros.

Moving Target Defense (MTD)

As técnicas de *Moving Target Defense* (MTD) procuram mitigar vulnerabilidades ao introduzir aleatoriedade contínua e dinâmica nas propriedades estruturais da aplicação em *runtime* [5] [10]. Ao introduzir aleatoriedade, como a adição de prefixos ou identificadores dinâmicos nas *tags* HTML, o MTD aumenta a incerteza dos atacantes [5]. Estas variações constantes tornam os *payloads* estáticos ineficazes, bloqueando injeções, sem depender de assinaturas ou regras tradicionais [5].

Isolamento/*Sandboxing*

Outra técnica utilizada é o *Sandboxing*, que permite ter controlo total das aplicações em execução, restringindo assim os privilégios de execução de componentes não confiáveis [5]. Desta forma, é possível impedir a propagação de ações maliciosas.

5 Avaliação Crítica

A persistência do XSS ao longo de quase três décadas não se explica por falta de conhecimento teórico – está bastante documentado e estudado. A principal razão reside no facto da *web* ter evoluído de forma a tornar o XSS mais difícil de eliminar, especialmente devido à introdução de novas tecnologias cada vez mais dinâmicas (e.g. SPAs, manipulação do DOM, dependência de APIs). Cada uma destas tecnologias acaba por introduzir novos pontos de injeção, que ainda podem não ter resposta e têm que ser estudados. Além disso, a maioria dos erros podia ser evitada, já que estes vêm de más práticas de desenvolvimento, pressão para entregar *software* rápido, falta de formação dos programadores ou pequenas falhas que passam despercebidas.

Do ponto de vista da mitigação e da defesa, existem diversas soluções úteis – cada uma com os seus prós e contras. Ainda assim, devido à existência de inúmeros tipos de XSS diferentes, dificilmente irá existir uma solução que resolva tudo sozinha. Contudo, existem ferramentas como o CSP, que, mesmo sendo uma camada de proteção muito forte, tem configurações muito complexas e, ao mínimo erro, pode não funcionar como desejado.

A abordagem mais eficaz para combater o XSS passa por aplicar uma defesa em camadas: uma implementação que segue o princípio de *secure by design*, a validação e sanitização em todas as camadas, a utilização das ferramentas corretas e testes de segurança contínuos ao longo do desenvolvimento. Acima de tudo, é indispensável investir na formação dos programadores, para que as boas práticas sejam aplicadas por hábito e não apenas por obrigação. O XSS continuará a existir enquanto a segurança for tratada como uma fase final do desenvolvimento, e não como uma propriedade essencial a cada linha de código escrita.

6 Conclusão

Apesar do *Cross-Site Scripting* ser uma ameaça já existente há mais de duas décadas, ainda continua a representar uma das vulnerabilidades mais relevantes no contexto das aplicações *web*. Esta relevância deve-se, em grande parte, à crescente complexidade das arquiteturas *web* modernas – nomeadamente a adoção generalizada de SPAs, AJAX e APIs assíncronas – que aumentam continuamente a superfície de ataque. A análise efetuada demonstra que a sua erradicação continua a ser um desafio complexo para a comunidade de cibersegurança, nomeadamente devido a falhas na validação, sanitização e tratamento seguro dos dados fornecidos pelos utilizadores.

A análise dos três tipos principais – *Stored*, *Reflected* e *DOM-based XSS* – revelou que, embora partilhem um ciclo de vida comum, diferem nos vetores de ataque e no grau de dificuldade de deteção. Em particular, *DOM-based XSS* destaca-se pela sua capacidade de contornar mecanismos de segurança tradicionais, pelo facto de acontecer exclusivamente no lado do cliente e sem interagir com o servidor.

As consequências para os utilizadores, sistemas e organizações são devastadoras – desde o roubo de dados sensíveis até danos financeiros –, o que pode ser comprovado pelos casos reais que analisámos na Subsecção 3.3. Foi possível observar que, independentemente do tamanho da organização, estas vulnerabilidades acontecem e podem ter impacto em milhões de utilizadores, o que acaba por afetar a reputação destas empresas.

No que diz respeito à mitigação, ficou evidente que nenhuma técnica isolada é suficiente. A combinação de práticas preventivas no desenvolvimento – como a validação por *whitelist*, a codificação de *output* e a utilização de técnicas de sanitização do lado do cliente – com mecanismos ativos em *runtime*, como a CSP e as técnicas de MTD, constitui uma abordagem de defesa em múltiplas camadas que a literatura científica reconhece como necessária.

Por fim, é importante reconhecer que o XSS é, em grande parte, uma vulnerabilidade evitável. A segurança *web* não pode ser tratada como uma etapa final no desenvolvimento, deve ser integrada desde a criação da aplicação, através de práticas como *secure by design*. Só através da utilização destas práticas em conjunto com ferramentas adequadas e a formação contínua dos programadores será possível reduzir de forma efetiva o risco associado a esta vulnerabilidade e proteger utilizadores, sistemas e organizações contra ataques cada vez mais sofisticados.

Referências

1. Alhamyani, R., Alshammari, M.: Machine learning-driven detection of cross-site scripting attacks. *Information* **15**(7) (2024). <https://doi.org/10.3390/info15070420>, <https://www.mdpi.com/2078-2489/15/7/420>
2. Alsaffar, M., Aljaloud, S., Mohammed, B.A., Al-Mekhlafi, Z.G., Almurayziq, T.S., Alshammari, G., Alshammari, A.: Detection of web cross-site scripting (xss)

- attacks. *Electronics* **11**(14) (2022). <https://doi.org/10.3390/electronics11142212>, <https://www.mdpi.com/2079-9292/11/14/2212>
3. Ayoubi, A., Laaouina, L., Jeghal, A., Tairi, H.: An improved detection of cross-site scripting (xss) attacks using a hybrid approach combining convolutional neural networks and support vector machine. *Journal of Cybersecurity and Privacy* **6**(1) (2026). <https://doi.org/10.3390/jcp6010018>, <https://www.mdpi.com/2624-800X/6/1/18>
 4. Gupta, S., Gupta, B.B.: Cross-site scripting (xss) attacks and defense mechanisms: classification and state-of-the-art. *International Journal of System Assurance Engineering and Management* **8**, 512 – 530 (2015), <https://api.semanticscholar.org/CorpusID:420462>
 5. Hannousse, A., Yahiouche, S., Nait-Hamoud, M.C.: Twenty-two years since revealing cross-site scripting attacks: A systematic mapping and a comprehensive survey. *Computer Science Review* **52**, 100634 (2024). <https://doi.org/10.1016/j.cosrev.2024.100634>, <https://www.sciencedirect.com/science/article/pii/S1574013724000182>
 6. Heiderich, M., Späth, C., Schwenk, J.: Dompurify: Client-side protection against xss and markup injection. In: Foley, S.N., Gollmann, D., Sneekenes, E. (eds.) *Computer Security – ESORICS 2017*. pp. 116–134. Springer International Publishing, Cham (2017)
 7. Hydera, I., Sultan, A.B.M., Zulzalil, H., Admodisastro, N.: Current state of research on cross-site scripting (xss) – a systematic literature review. *Information and Software Technology* **58**, 170–186 (2015). <https://doi.org/10.1016/j.infsof.2014.07.010>, <https://www.sciencedirect.com/science/article/pii/S0950584914001700>
 8. Kaur, J., Garg, U., Bathla, G.: Detection of cross-site scripting (xss) attacks using machine learning techniques: a review. *Artificial Intelligence Review* **56**(11), 12725–12769 (2023)
 9. Pasini, S., Maragliano, G., Kim, J., Tonella, P.: Cross-site scripting adversarial attacks based on deep reinforcement learning: Evaluation and extension study. *Journal of Systems and Software* **237**, 112856 (2026). <https://doi.org/10.1016/j.jss.2026.112856>, <https://www.sciencedirect.com/science/article/pii/S0164121226000907>
 10. Rodríguez, G.E., Torres, J.G., Flores, P., Benavides, D.E.: Cross-site scripting (xss) attacks and mitigation: A survey. *Computer Networks* **166**, 106960 (2020). <https://doi.org/10.1016/j.comnet.2019.106960>, <https://www.sciencedirect.com/science/article/pii/S1389128619311247>
 11. Singh, M., Singh, P., Kumar, P.: An analytical study on cross-site scripting. In: *2020 International Conference on Computer Science, Engineering and Applications (ICCSEA)*. pp. 1–6 (2020). <https://doi.org/10.1109/ICCSEA49143.2020.9132894>
 12. Stuttard, D., Pinto, M.: *The Web Application Hacker’s Handbook: Finding and Exploiting Security Flaws*. Wiley, 2 edn. (2011)
 13. Younas, F., Raza, A., Thalji, N., Abualigah, L., Zitar, R.A., Jia, H.: An efficient artificial intelligence approach for early detection of cross-site scripting attacks. *Decision Analytics Journal* **11**, 100466 (2024). <https://doi.org/10.1016/j.dajour.2024.100466>, <https://www.sciencedirect.com/science/article/pii/S2772662224000705>